

Can policy gradient replicate the exact soccer solver?

REINFORCE, A2C and PPO, trained by self-play and by fictitious play, scored against the exact solution of the discounted 100-step soccer game. Every number below is read from the experiment files in the repository.

Summary

- Six learners, REINFORCE, A2C and PPO each trained by self-play and by fictitious play, on the discounted 100-step soccer game, scored against the exact solution.
- Their exploitability ranges from 0.76 to 0.95; the exact solution is 0.
- Fictitious-play training wins more often against a random player; self-play training ties the exact equilibrium more often.
- The two players' policies are not mirror images (mirror gap 0.44 to 0.81; 0 for the exact solution).

Results are for 3 seeds at one untuned budget. The action probabilities the trained policies output are in section 6.

1. What was done

These choices fix the setup used throughout.

topic	what was done	why
Environment	The deterministic A10 board is the main environment; the random move-order board is kept as an earlier comparison.	The deterministic board has no mixed-equilibrium stages, so the comparison with the exact solution is clean.
Objective	Discount 0.9 over 100 steps, a tie at the end, and the remaining step count as a network input.	Policy gradient can only approximate a finite-horizon discounted reward.
Fictitious play	Each player is trained by policy gradient against the average of the opponent's past policies. Matrix-level fictitious play is kept only as a side check (section 8).	Policy gradient does the solving; no game is solved explicitly.
Win rate	1,000 repeated games from the kickoff against a random player, the exact Nash policy and the exact best response; wins, ties and losses are all counted.	Repeated play and counting wins.

2. Setup

The environment is the deterministic A10 soccer game on a 7×5 board with a three-cell goal; it has no mixed-strategy stage games, which keeps the comparison with the exact solution clean. The objective is **discounted** ($\gamma = 0.9$) over a **100-step horizon**, a tie if nobody scores, and the network receives the **remaining step count** as an input. The reference is the exact discounted backward induction over the same 100 steps (`soccer_nash/finite_horizon.py`), whose equilibrium has exploitability 0 and ties itself every game.

3. The learners

algorithm	update
REINFORCE	plain discounted returns, no baseline
A2C	10-step bootstrapped advantage from a learned critic
PPO	GAE ($\lambda = 0.95$), clipped ratio 0.2, 4 epochs

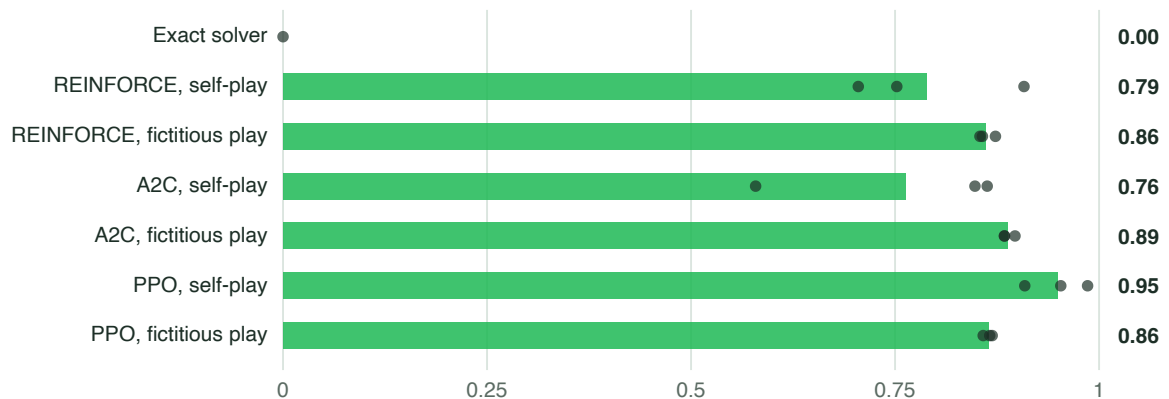
Self-play trains both current networks against each other. **Fictitious play** is how policy gradient solves the game here, with no matrix solving: each player keeps improving its own network by policy gradient against the *empirical average* of the opponent's past policies (frozen snapshots, one drawn per episode), and reports the average of its own snapshots. All learners use learning rate 10^{-3} , 64 episodes per batch, 2,000 iterations, an entropy bonus of 0.01, and a 64×64 network; none was tuned.

4. How they are scored

- **Exploitability**: the discounted value both exact best responses earn from the kickoff, summed. Zero exactly at an equilibrium.
- **Wins, ties and losses**: counts over 1,000 repeated games from the kickoff, against a random player, the exact Nash policy and the exact best response.
- **Mirror gap**: how far the two players' policies are from being mirror images (flip the board, swap the players), averaged over all states at five points in the game.

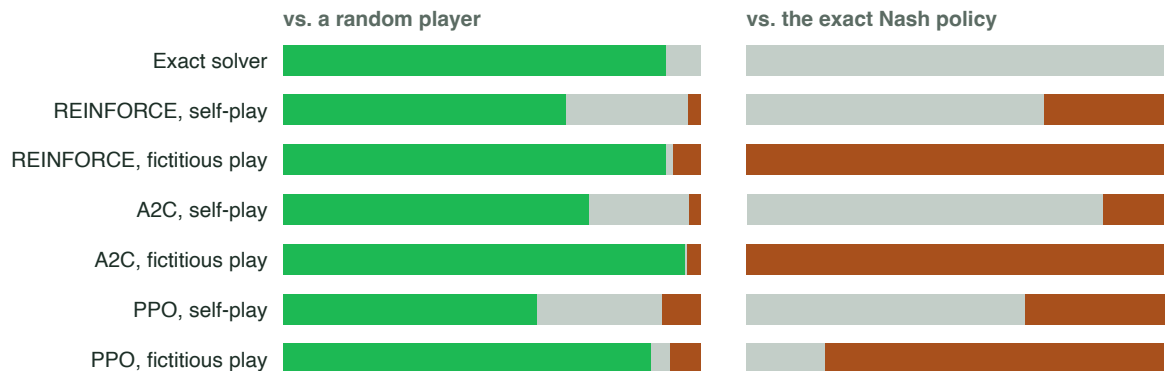
5. Results

learner	exploitability (mean $\pm$ sd)	W/T/L vs. random	vs. exact Nash	vs. best response	mirror gap
Exact solver	0.000 $\pm$ 0.000	0.92/0.08/0.00	0.00/1.00/0.00	0.00/1.00/0.00	0.00
REINFORCE, self-play	0.788 $\pm$ 0.106	0.68/0.29/0.03	0.00/0.71/0.29	0.00/0.00/1.00	0.66
REINFORCE, fictitious play	0.861 $\pm$ 0.010	0.92/0.02/0.07	0.00/0.00/1.00	0.02/0.00/0.98	0.44
A2C, self- play	0.763 $\pm$ 0.160	0.73/0.24/0.03	0.00/0.85/0.15	0.02/0.00/0.98	0.53
A2C, fictitious play	0.888 $\pm$ 0.008	0.96/0.00/0.03	0.00/0.00/1.00	0.01/0.00/0.99	0.44
PP0, self- play	0.949 $\pm$ 0.039	0.61/0.30/0.09	0.00/0.67/0.33	0.00/0.00/1.00	0.81
PP0, fictitious play	0.864 $\pm$ 0.006	0.88/0.05/0.07	0.00/0.19/0.81	0.00/0.00/1.00	0.48



Exploitability by learner: bar = mean, dots = individual seeds.

win tie loss



Two patterns repeat. Fictitious-play training beats a random player more often (0.88 to 0.96) but loses almost every game to the exact equilibrium. Self-play training wins less often against random and ties

the equilibrium more often: it plays more cautiously. Neither is close to equilibrium play, and every learner loses almost every game to the exact best response.

Does more training help?

Self-play with **8,000** iterations instead of 2,000 (2 seeds per learner; every other setting unchanged). The fictitious-play runs were too slow to repeat at this length.

learner	exploitability, 2,000 it. (3-seed mean)	exploitability, 8,000 it. (each seed)	W/T/L vs. random (8,000 it.)	W/T/L vs. exact Nash (8,000 it.)
REINFORCE, self-play	0.788	0.49 / 0.91	0.91/0.08/0.02	0.00/0.38/0.62
A2C, self- play	0.763	0.44 / 0.50	0.79/0.20/0.01	0.00/0.79/0.21
PPO, self- play	0.949	0.91 / 0.96	0.97/0.01/0.02	0.00/0.00/1.00

REINFORCE ends clearly lower on 1 of 2 seeds; A2C ends clearly lower on 2 of 2 seeds; PPO ends clearly lower on 0 of 2 seeds; the best single run reaches 0.44, still far from 0. With two seeds per learner this is a trend, not a result.

Fictitious play with best-response phases

A stricter version of fictitious play: in each round each player runs 100 (or 300) policy-gradient iterations to approximate a best response to the average of the opponent's earlier best responses, then adds that policy to its history. The reported policy is the per-state average of the best responses. Exploitability after each checkpoint round (1 seed per run, 64 episodes per iteration):

run	round 5	round 10	round 15	round 20	round 25	round 30
A2C, 100 iterations per best response	0.85	0.88	0.89	0.90	0.91	0.92
A2C, 300 iterations per best response	0.89	0.92	0.93			
PPO, 100 iterations per best response	0.89	0.92	0.90	0.90	0.91	0.91
REINFORCE, 100 iterations per best response	0.85	0.85	0.87	0.86	0.87	0.87

In every run the exploitability at the last checkpoint is higher than at the first; none comes close to 0.

6. Action probabilities

What the trained policies actually output: the probability of each move (U, D, L, R) for both players, at the start of the game (all 100 steps left), at four fixed positions. The most likely move is in bold. The learners are seed 0 of the runs in section 5; the exact solver's play is shown for comparison.

Kickoff: player 0 at (0, 1), player 1 at (6, 3), player 0 has the ball

policy	player 0				player 1			
	U	D	L	R	U	D	L	R
Exact solver	100%	0%	0%	0%	100%	0%	0%	0%
REINFORCE, self-play	46%	22%	31%	0%	7%	1%	57%	35%
REINFORCE, fictitious play	41%	1%	29%	29%	1%	17%	82%	1%
A2C, self-play	31%	26%	33%	10%	9%	1%	69%	21%
A2C, fictitious play	62%	0%	13%	25%	1%	8%	90%	1%
PP0, self-play	100%	0%	0%	0%	0%	100%	0%	0%
PP0, fictitious play	93%	0%	0%	7%	0%	3%	96%	0%

Players side by side in a goal row: player 0 at (3, 2), player 1 at (4, 2), player 0 has the ball

policy	player 0				player 1			
	U	D	L	R	U	D	L	R
Exact solver	100%	0%	0%	0%	0%	0%	0%	100%
REINFORCE, self-play	88%	3%	6%	2%	4%	31%	31%	34%
REINFORCE, fictitious play	5%	1%	2%	92%	25%	32%	42%	1%
A2C, self-play	26%	36%	28%	10%	7%	5%	18%	69%
A2C, fictitious play	20%	1%	3%	76%	37%	57%	5%	1%
PP0, self-play	0%	0%	0%	100%	0%	0%	0%	100%
PP0, fictitious play	79%	0%	0%	21%	35%	5%	59%	0%

Carrier near the goal, defender far away: player 0 at (5, 1), player 1 at (6, 3), player 0 has the ball

policy	player 0				player 1			
	U	D	L	R	U	D	L	R
Exact solver	0%	0%	0%	100%	0%	100%	0%	0%
REINFORCE, self-play	90%	4%	5%	1%	0%	100%	0%	0%
REINFORCE, fictitious play	11%	1%	3%	86%	1%	95%	3%	1%
A2C, self-play	25%	39%	25%	11%	0%	97%	0%	3%
A2C, fictitious play	31%	0%	4%	65%	1%	95%	3%	1%
PP0, self-play	0%	0%	0%	100%	0%	100%	0%	0%
PP0, fictitious play	91%	0%	0%	9%	0%	99%	1%	0%

The right player holds the ball, players stacked: player 0 at (3, 2), player 1 at (3, 3), player 1 has the ball

policy	player 0				player 1			
	U	D	L	R	U	D	L	R
Exact solver	100%	0%	0%	0%	0%	0%	100%	0%
REINFORCE, self-play	3%	3%	91%	3%	0%	0%	99%	1%
REINFORCE, fictitious play	14%	2%	15%	69%	1%	2%	96%	0%
A2C, self-play	4%	1%	71%	24%	0%	14%	84%	1%
A2C, fictitious play	6%	2%	38%	53%	1%	2%	97%	0%
PP0, self-play	0%	0%	0%	100%	0%	100%	0%	0%
PP0, fictitious play	6%	0%	0%	94%	0%	0%	99%	0%

Rows that put nearly all the probability on one move are close to deterministic play; the exact solution is deterministic at these positions.

7. Earlier experiments

Earlier, the same algorithms were run on the stationary discounted game (no step count), from the kickoff or from random starting states.

setup	algorithm	exploitability	mean equilibrium regret	action agreement
A10 board, stationary, kickoff starts	REINFORCE	0.83	0.124	0.28
A10 board, stationary, kickoff starts	A2C	0.83	0.122	0.29
A10 board, stationary, kickoff starts	PP0	0.87	0.167	0.25
random board, stationary, kickoff starts	REINFORCE	0.73	0.056	0.44
random board, stationary, kickoff starts	A2C	0.78	0.058	0.45
random board, stationary, kickoff starts	PP0	0.85	0.108	0.32
random board, stationary, exploring starts	REINFORCE	0.80	0.027	0.71
random board, stationary, exploring starts	A2C	0.82	0.030	0.70
random board, stationary, exploring starts	PP0	0.91	0.072	0.51

Exploring starts raise per-state accuracy (action agreement) but not play from the kickoff.

8. Fictitious play on the exact stage games

As a side check, fictitious play was also run on the exact 4×4 stage matrices of the random move-order board (2,380 states, 94 with no pure saddle). On rock-paper-scissors best-response dynamics cycle forever while fictitious play settles at 1/3 each; on the soccer stage games it recovers the exact values.

rounds	mean stage-game gap	max stage-game gap	max value error
100	0.00233	0.02020	0.01010
1000	0.00026	0.00296	0.00133
10000	0.00004	0.00094	0.00037

This is matrix-level fictitious play. Here policy gradient itself does the solving, which is what section 5 tests.